Middle East Technical University
Department of Mechanical Engineering
ME310 Numerical Methods

Iterative Solvers
An Application to a Solution of a PDE

Assistant: Atakan Aygün
atakana@metu.edu.tr

Systems of linear algebraic equations arise naturally in many areas of mechanical engineering. In computational analysis, physical problems are typically described by differential equations that model conservation laws such as conservation of mass, momentum, and energy. When these governing equations are discretized for numerical solution, they lead to systems of the form

$$A\mathbf{x} = \mathbf{b}$$

where A is a matrix representing the discretized operator, $\mathbf{x}$ is the vector of unknown variables, and $\mathbf{b}$ contains the source terms and boundary conditions.

Such linear systems appear in a wide variety of mechanical engineering applications. For example, in heat transfer problems the unknown vector may represent temperatures at discrete points in a solid. In computational fluid dynamics, the unknowns may correspond to pressure or velocity values at grid points. In solid mechanics, displacement components at nodes of a discretized structure form the unknown vector. Regardless of the physical context, the numerical solution of these problems ultimately reduces to solving a linear system.

For realistic engineering simulations, the number of unknowns can become very large. Even a simple two–dimensional discretization of a physical domain may lead to thousands or millions of equations. The matrices that arise from such discretizations are typically large, sparse, and structured, reflecting the local interactions between neighboring points in the physical domain. Because of the size of these systems, classical direct methods such as Gaussian elimination may become computationally expensive in terms of both memory and execution time.

For this reason, iterative methods play a central role in numerical engineering analysis. Instead of computing the solution in a finite sequence of elimination steps, iterative methods begin with an initial guess and successively improve the solution until a desired accuracy is reached. Classical methods such as the Jacobi, Gauss–Seidel, and Successive Over-Relaxation (SOR) methods form the foundation of many modern numerical solvers used in large-scale engineering simulations.

To study the behavior and performance of these iterative methods, it is useful to consider a representative model problem that produces the types of linear systems commonly encountered in engineering applications. In this note, we use the Poisson equation as such a model problem. The Poisson equation appears in several mechanical engineering contexts, including steady heat conduction with internal heat generation, pressure correction equations in incompressible fluid flow, and potential formulations of elasticity. When discretized on a computational grid, it produces a sparse linear system with a regular structure, making it particularly suitable for illustrating the convergence properties and efficiency of iterative solution techniques.

The Two–Dimensional Poisson Equation

In two spatial dimensions, the Poisson equation can be written as

$$\nabla^2 u(x, y) = f(x, y), \qquad (x, y) \in \Omega,$$

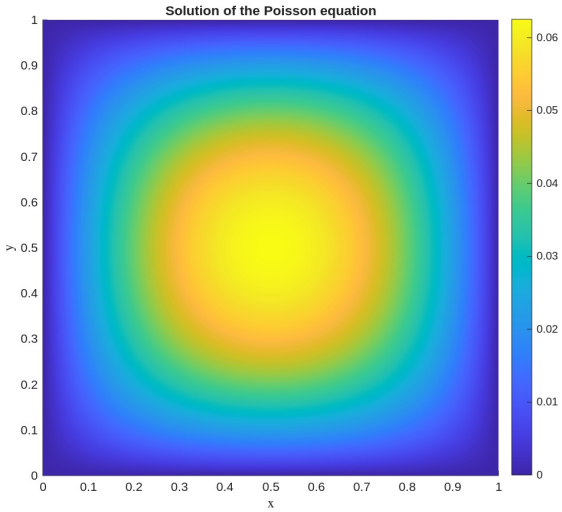

Figure 1: Solution of the Poisson equation with homogeneous boundary conditions and exact solution of $u_e = (x - x^2)(y - y^2)$.

where $u(x, y)$ is the unknown scalar field, $f(x, y)$ is a given source term, and Ω denotes the computational domain. The operator ∇^2 is the Laplacian operator, which in Cartesian coordinates is defined as

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}.$$

In this report, we consider the Poisson equation on a square domain

$$\Omega = [0, 1] \times [0, 1].$$

To obtain a well-posed problem, appropriate boundary conditions must be specified on the boundary $\partial\Omega$. For simplicity, homogeneous boundary conditions are prescribed,

$$u(x, y) = 0, \qquad (x, y) \in \partial\Omega,$$

We will use a forcing term which yields an exact solution of

$$u_e = (x - x^2)(y - y^2).$$

The exact solution is visualized in Figure 1. The Poisson equation serves as a convenient model problem for numerical methods because its structure is simple while still capturing the essential features of many engineering applications. When discretized using finite difference methods on a structured grid, the equation leads to a sparse system of linear algebraic equations that can be efficiently solved using iterative techniques.

The domain Ω is split into N points in each x and y directions. A sample domain can be seen in Figure 2.

Since the calculations are performed for the inner nodes (the unknowns), boundary nodes are excluded when forming the coefficient matrix. Discretization of the Poisson equation is performed with the central finite difference method which can be seen in Equation 1.

$$\frac{\partial^2 u}{\partial x^2} = \frac{u(x + h) - 2u(x) + u(x - h)}{h^2}, \tag{1}$$

where h is the spacing between the nodes. This is a similar approach that is performed in ME311 Heat Transfer course. Assuming a uniform spacing ($\Delta x = \Delta y = h$), and using the same approach for

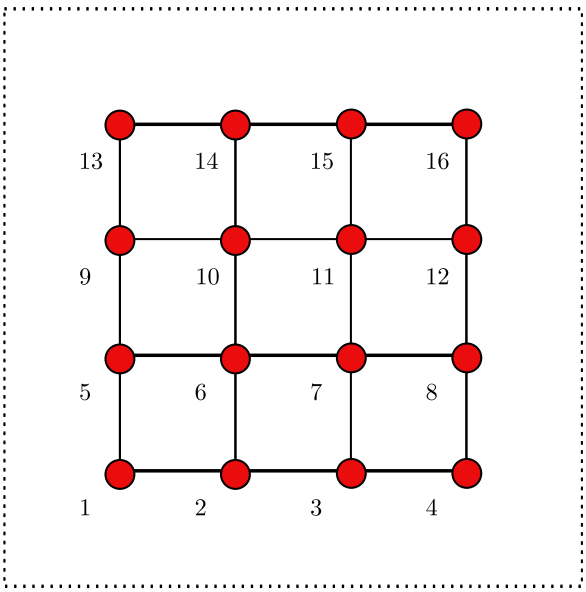

Figure 2: A sample grid and the numbering of nodes for a domain divided into 4 points in each direction.

discretizing the derivative with respect to y, the coefficient matrix $\mathbf{A}$ becomes:

$$\mathbf{A} = \begin{bmatrix} \mathbf{D} & -\mathbf{I} & 0 & 0 & 0 & \dots & 0 \\ -\mathbf{I} & \mathbf{D} & -\mathbf{I} & 0 & 0 & \dots & 0 \\ 0 & -\mathbf{I} & \mathbf{D} & -\mathbf{I} & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & -\mathbf{I} & \mathbf{D} & -\mathbf{I} & 0 \\ 0 & \dots & 0 & 0 & -\mathbf{I} & \mathbf{D} & -\mathbf{I} \\ 0 & \dots & 0 & 0 & 0 & -\mathbf{I} & \mathbf{D} \end{bmatrix},$$

where $\mathbf{I}$ is the identity matrix, and $\mathbf{D}$ is,

$$\mathbf{D} = \begin{bmatrix} 4 & -1 & 0 & 0 & 0 & \dots & 0 \\ -1 & 4 & -1 & 0 & 0 & \dots & 0 \\ 0 & -1 & 4 & -1 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & -1 & 4 & -1 & 0 \\ 0 & \dots & 0 & 0 & -1 & 4 & -1 \\ 0 & \dots & 0 & 0 & 0 & -1 & 4 \end{bmatrix}.$$

Here, the boundary nodes are masked since they are equal to zero and we will only solve for the inner nodes. In Figure 3, the sparsity of matrix A is presented with $N = 5$. You can use `spy(A)` command in Matlab to visualize sparsity patterns of matrices. It can be seen that the matrix is banded and symmetric and its size is always $N^2 \times N^2$.

The Poisson equation is a very common equation and therefore there are some built-in commands in Matlab to build the coefficient matrix such as `gallery` for a standard matrix or `delsq` for different domains. However, in the supporting codes, you will see the matrix is built from scratch for a more clear insight about the matrix.

Numerical Results and Behavior of Different Linear Solvers

In this section, the behavior of Jacobi, Gauss-Seidel and successive over relaxation methods for the two-dimensional Poisson equation is analyzed. Since the coefficient matrix $\mathbf{A}$ is strictly diagonally

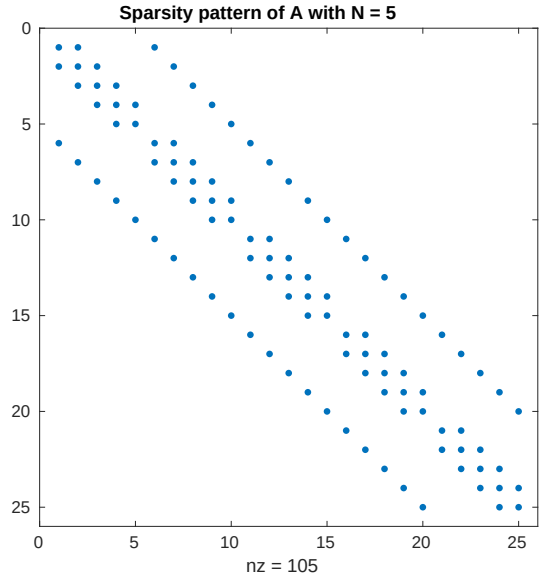

Figure 3: Sparsity pattern of matrix A with $N = 5$. Number of nonzeros is 105.

Table 1: Minimum eigenvalues for different sizes of the matrix $\mathbf{A}$.

N	λ_{min}
5	0.5359
10	0.1620
20	0.0447
50	0.0076
100	0.0019

dominant, these methods are suitable for this test case. We can also check if the matrix is symmetric and positive-definite for a guaranteed convergence of the Gauss-Seidel method.

You can check if a matrix is symmetric by using the Matlab command `issymmetric`, or check the values of $\mathbf{A} - \mathbf{A}^T$ since they should be equal. Our coefficient matrix is full of integers and this operation will yield full of zero entries. For the positive definiteness we can check if the eigenvalues of the matrix are strictly positive. The Matlab command `eig(A)` gives us all of the eigenvalues of a matrix. You can see the minimum eigenvalues of the coefficient matrix for different sizes in Table 1. This simple matrix is positive definite and we can use the Gauss-Seidel method here.

For the iterative algorithms, you can define different stopping criteria. Most common ones are defining a maximum number of iterations or a stopping tolerance of error or residual norms. Since we know the exact solution of this problem, we can define the true error norm $||u - u_e||_\infty$ as the stopping criteria along with a maximum number of iterations. The algorithms will stop if they achieve the maximum number of iterations, or the error norm drops below a tolerance value before that. I want to set a tolerance value of 1×10^{-3} because since the Jacobi and Gauss-Seidel methods converge slowly for large N values. We also set a maximum number of iterations of 5000 to stop primitively if an algorithm takes too many iterations to converge.

We perform tests with a set of $N = \{20, 50, 100\}$ to see the convergence rate and time it takes for convergence of different iterative methods. For the successive over relaxation method the relaxation parameter w changes the convergence behavior. For the following results an optimal parameter $w_{opt} = \frac{2}{1+\sin(\pi h)}$ is used [1]. For $N = 20$, the convergence rate of different solvers is shown in Figure 4. The SOR method error norm drops down to the tolerance very rapidly compared to the Jacobi and Gauss-Seidel method since the main reason of the relaxation parameter is to improve the convergence rate.

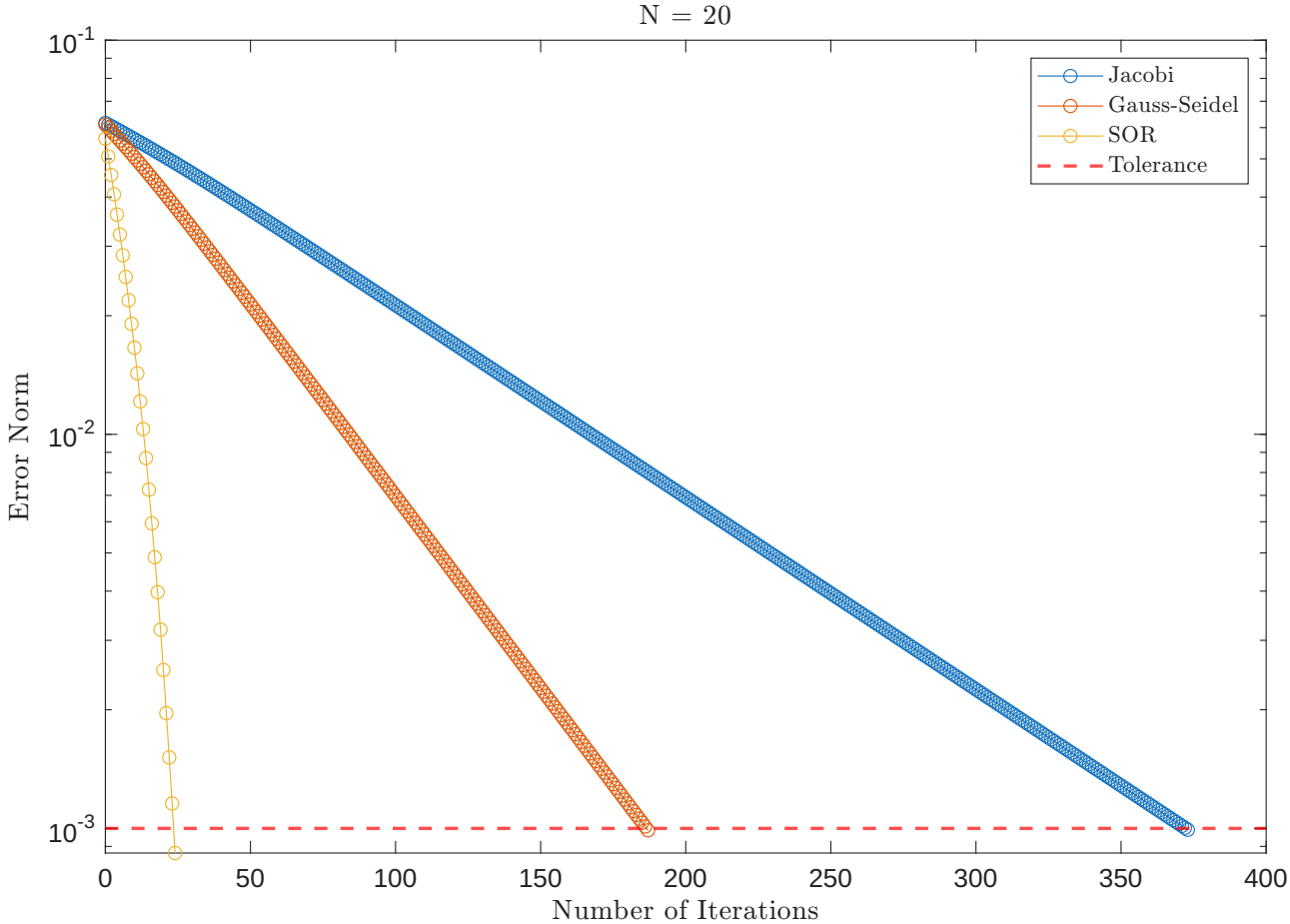

Figure 4: History of the error norm for different iterative methods with $N = 20$.

Table 2: Total number of iterations and the last error norms for different sizes of the coefficient matrix.

Method	N	$\|\|u - u_e\|\|_\infty$	Number of Iterations
Jacobi	20	0.0009	374
	50	0.0009	2211
	100	0.0059	5000
Gauss-Seidel	20	0.0009	189
	50	0.0009	1108
	100	0.0009	4340
SOR	20	0.0009	25
	50	0.0009	60
	100	0.0009	119

Table 3: Computational time in seconds for different iterative methods, and different matrix size N.

N	Jacobi	G-S	SOR	MATLAB(\)
20	0.162	0.067	0.024	0.001
50	81.317	41.313	2.593	0.077
60	381.513	192.813	5.727	0.187

The total number of iterations to achieve an error norm below the tolerance for different sizes of the coefficient matrix is given in Table 2. The SOR method again dominates in the performance compared to the other methods. Also for a size of $N = 100$, the Jacobi method cannot reach the desired error norm tolerance before it hits the maximum number of iterations.

It is generally a good practice to look for the timings of different algorithms to reach the desired tolerance values. For the timings, we also include the performance of the default solver of Matlab, `mldivide` or the backslash operator (\) also. The timings in seconds are presented in Table 3. As the matrix size grows The time it takes for convergence drastically increases. The default solver is very fast compared to the iterative algorithms used in this note. This function performs a series of checks to the input matrix to choose the best calculation method. Based on these checks, it can use a triangular or permuted triangular solver, Hessenberg solver, LU solver, or Cholesky solver. The timings of the iterative solvers increase because these methods checks every entry of the matrix $\mathbf{A}$ with the naive implementation. Most of the entries here are 0 and we perform redundant operations and store the zero entries unnecessarily.

Table 4: Error norms and maximum number of iterations of SOR method with different relaxation parameters for $N = 50$.

w	$\|\|u - u_e\|\|_\infty$	Number of iterations	Time (s)
0.30	0.0023	5000	201.315
0.70	0.0009	2056	82.754
1.20	0.0009	738	30.503
1.60	0.0009	275	11.574
1.88	0.0009	60	2.598
2.00	0.0574	5000	204.170

The relaxation parameter has a huge impact on the convergence of SOR method. There are some studies for the optimal relaxation parameter as we have used in this note. Choosing a good relaxation value is important for different problem setups. For our test case, the behavior of the solver with different relaxation parameters can be seen in Table 4. For a very low under relaxation, the solver did not converge at all and reach the maximum number of iterations. Using the optimal parameter from [1] is extremely useful compared to the other parameters. Going above this parameter by over relaxing the system more is not very useful as seen in the Table 4.

References

[1] S. Yang and M. K. Gobbert, "The optimal relaxation parameter for the SOR method applied to the Poisson equation in any space dimensions," *Applied Mathematics Letters*, vol. 22, no. 3, pp. 325–331, 2009.